Mental Health Assessment and Analysis System

21CSC205P Database Management Systems

A MINI PROJECT REPORT

Submitted by

DIA GARG [RA2411026010159]
DEBOBRITA DAS [RA2411026010143]

Under the Guidance of

Dr. N. Snehalatha

Associate Professor, Department of Computational Intelligence

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in AI/ML

DEPARTMENT OF NETWORKING AND COMMUNICATIONS
SCHOOL OF COMPUTING
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that Project report titled "**Mental Health Assessment and Analysis System**" is the bonafide work of "**DIA GARG [RA2411026010159], DEBOBRITA DAS [RA2411026010143]"** who carried out the **21CSC205P Database Management Systems** mini project work under my supervision.

Signature
COURSE HANDLING FACULTY
Dr. N. Snehalatha
Associate Professor
Department of Computational Intelligence
SRM Institute of Science and Technology
Kattankulathur

Signature
HEAD OF THE DEPARTMENT
Dr. Annie Uthra R
Head of the Department
Professor and Head
Department of Computational Intelligence
SRM Institute of Science and Technology
Kattankulathur

ABSTRACT

The Mental Health Assessment and Analysis System is a database-driven application designed to collect user responses to psychological assessment questions, calculate total scores, and determine mental health patterns based on predefined score ranges. The system uses ER modeling and relational database concepts to manage users, assessments, responses, and result patterns efficiently.

The project demonstrates the implementation of database design principles including ER modeling, relational schema design, normalization techniques, SQL queries, triggers, transactions, concurrency control, and recovery mechanisms. The system ensures data integrity, consistency, and efficient retrieval of assessment results.

This project highlights the practical application of DBMS concepts in building a structured and normalized database system for real-world assessment analysis.

TABLE OF CONTENTS

Chapter No.	Title	Page no.
	ABSTRACT	3
1	Problem understanding, Identification of Entity and Relationships, Construction of DB using ER Model for their project	7
2	Design of Relational Schemas, Creation of Database and Tables for their project	12
3	3.1Writing queries based on the concepts of Aggregate functions, constraints, sets	20
	3.2 Writing complex queries based on subqueries, joins and views	23
	3.3 Writing complex queries based on the concepts of functions, Triggers, Cursors and Exception Handling	27
4	4.1 Analysing the pitfalls, identifying the dependencies, and applying Normalizations- 1NF, 2NF, 3NF 4.2 Analysing the pitfalls, identifying the dependencies, and applying Normalizations- 4NF, 5NF	
5	Implementation of concurrency control and recovery mechanisms.	
6	Front-end And Back-End Implementation code	
7	Results and Discussions	

LIST OF FIGURES

Figure No.	Name	Page no.
1	Figure 1.1: ER Diagram of Mental health Assessment and Analysis System	11
2	Figure 2.1: Relational Schema for Mental Health Assessment and Analysis System	12
3	Figure 3.1.1: Error message indicating violation of CHECK constraint.	19
4	Figure 3.1.2: Record inserted successfully.	19
5	Figure 3.1.3: Error due to UNIQUE constraint violation.	20
6	Figure 3.2.1: Displays average total score	20
7	Figure 3.2.2: Displays highest score.	21
8	Figure 3.2.3: Displays lowest total score.	21
9	Figure 3.3.1: List of unique user IDs	22
10	Figure 3.3.2: List of user IDs including duplicates	22
11	Figure 3.3.3: Common user IDs	23
12	Figure 3.4.1: Above-average assessments	23
13	Figure 3.4.2: Users who attempted test	24
14	Figure 3.4.3: Highest scoring assessment	24
15	Figure 3.5.1: User names with scores	25
16	Figure 3.5.2: User responses with questions	25
17	Figure 3.5.3: Assessment classification	26
18	Figure 3.6.1: View output(user_scores)	27
19	Figure 3.7.1:Updated total score after trigger	28
20	Figure 3.8.1: Cursor output	29

LIST OF TABLES

Figure No.	Name	Page no.
1	Table 2.1: User	13
2	Table 2.1: Responses	13
3	Table 2.3: Question	14
4	Table 2.4: Assessment	14
5	Table 2.5: Mental Health Reference	15

CHAPTER 1

PROBLEM UNDERSTANDING, IDENTIFICATION OF ENTITY AND RELATIONSHIPS, CONSTRUCTION OF DB USING ER MODEL FOR THE Mental Health Assessment and Analysis System

1.1 Introduction

Mental health assessment is an important process for evaluating an individual's psychological and emotional well-being. With increasing academic, professional, and social pressures, there is a growing need for structured systems that can collect and analyze assessment data accurately. Manual methods of recording responses are often inefficient, prone to errors in score calculation, and lack proper organization for long-term data management. A database-driven approach provides a systematic solution for storing, processing, and retrieving assessment information in a consistent and reliable manner.

The proposed *Mental Health Assessment and Analysis System* focuses on designing and implementing a relational database that records user responses to predefined mental health questions, calculates total scores, and classifies results based on predefined score ranges. The project emphasizes the application of core Database Management System (DBMS) concepts such as Entity-Relationship modeling, relational schema design, normalization, and SQL implementation. The system ensures data integrity, eliminates redundancy, and enables efficient retrieval of results, thereby demonstrating the practical use of structured database design in an assessment-based application.

1.2 Motivation

The motivation behind this project is to design a structured and efficient database system that eliminates the limitations of manual mental health assessment processes. Manual methods often lead to data redundancy, calculation errors, and difficulty in maintaining consistent records. By implementing a relational database system, assessment data can be stored systematically, ensuring accuracy in score computation and reliable classification of mental health patterns. Additionally, this project provides an opportunity to apply fundamental Database Management System concepts such as ER modeling, relational schema design, normalization, and SQL implementation in a practical and real-world context. The system aims to demonstrate how structured data management can enhance reliability, consistency, and efficiency in assessment-based applications.

1.3 Scope

The scope of the Mental Health Analyzer is limited to database design and implementation using DBMS principles.
The scope includes:

- User registration and management.
- Question storage.
- Recording responses.
- Automatic score calculation.
- Mapping total score to mental health category.
- Maintaining normalized database structure.

The project primarily focuses on backend database design, which serves as the foundation for future system enhancements.

1.4 Problem Statement

The primary problem addressed in this project is the lack of a structured and reliable system for storing, managing, and analyzing mental health assessment data. Manual or unorganized methods of recording responses often result in data redundancy, inconsistencies, and errors in score calculation. Furthermore, without a properly designed database, it becomes difficult to maintain relationships between users, assessments, responses, and result classifications. This creates challenges in ensuring data integrity and efficient retrieval of information.

The objective of this project is to design and implement a relational database system that systematically records user responses to predefined mental health questions, calculates total assessment scores, and maps those scores to predefined mental health patterns. The system must maintain referential integrity using primary and foreign keys, follow normalization principles to eliminate redundancy, and support efficient query processing. The project aims to demonstrate the practical application of DBMS concepts in building a structured and consistent assessment-based data management system.

1.5 Project Requirements

The proposed *Mental Health Assessment and Analysis System* requires the development of a structured relational database capable of storing, managing, and analyzing assessment data efficiently. The system must ensure proper organization of data entities and maintain integrity through well-defined relationships. The requirements of the project can be categorized as follows:

Functional Requirements

The system must support the following operations:

- Storage of user details with a unique identifier for each user.
- Maintenance of predefined mental health assessment questions.
- Recording of user responses for each assessment attempt.
- Calculation and storage of total assessment scores.
- Classification of assessment results based on predefined score ranges.

Non-Functional Requirements

In addition to functional capabilities, the system must satisfy the following quality requirements:

- Enforcement of primary key and foreign key constraints to maintain referential integrity.
- Prevention of invalid or duplicate data entries.
- Maintenance of consistency during score calculation and data storage.
- Efficient retrieval of assessment results through structured SQL queries.

The database design must follow normalization principles to eliminate redundancy and ensure logical data organization. The overall implementation should demonstrate the practical application of core DBMS concepts such as ER modeling, relational schema design, and structured query execution.

1.6 Identification of Entity and Relationships

Entities:

- USER
- ASSESSMENT
- QUESTION
- RESPONSE
- MENTAL_HEALTH_PATTERN

Relationships:

- USER takes ASSESSMENT
- ASSESSMENT contains RESPONSE

- QUESTION answered in RESPONSE
- ASSESSMENT generates MENTAL_HEALTH_PATTERN

Cardinality:

- One USER → many ASSESSMENTS (1:M)
- One ASSESSMENT → many RESPONSES (1:M)
- One QUESTION → many RESPONSES (1:M)

1.7 Construction of DB Using ER Model for the Mental Health Assessment and Analysis System

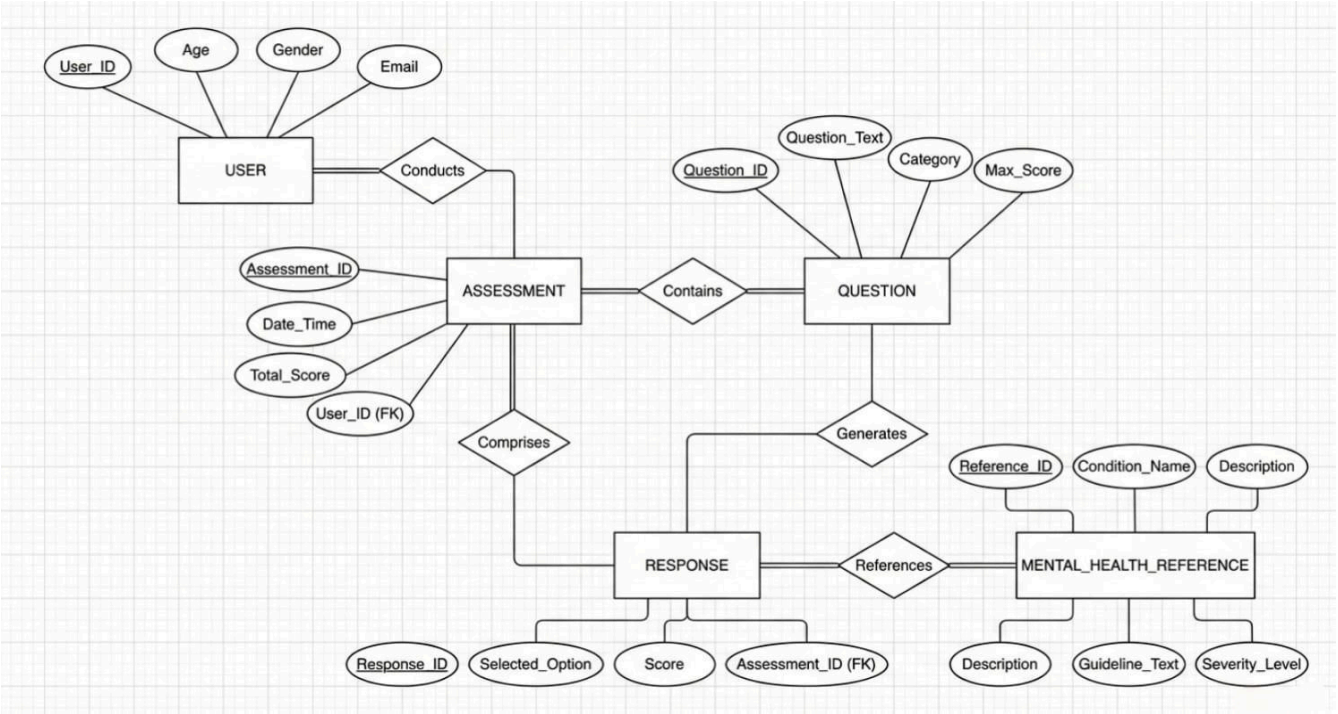

Figure 1.1: ER Diagram of Mental health Assessment and Analysis System

Chapter 2

Design of Relational Schemas, Creation of Database and Tables for Mental Health Assessment and Analysis System

2.1 Relational Schema for Mental Health Assessment and Analysis System

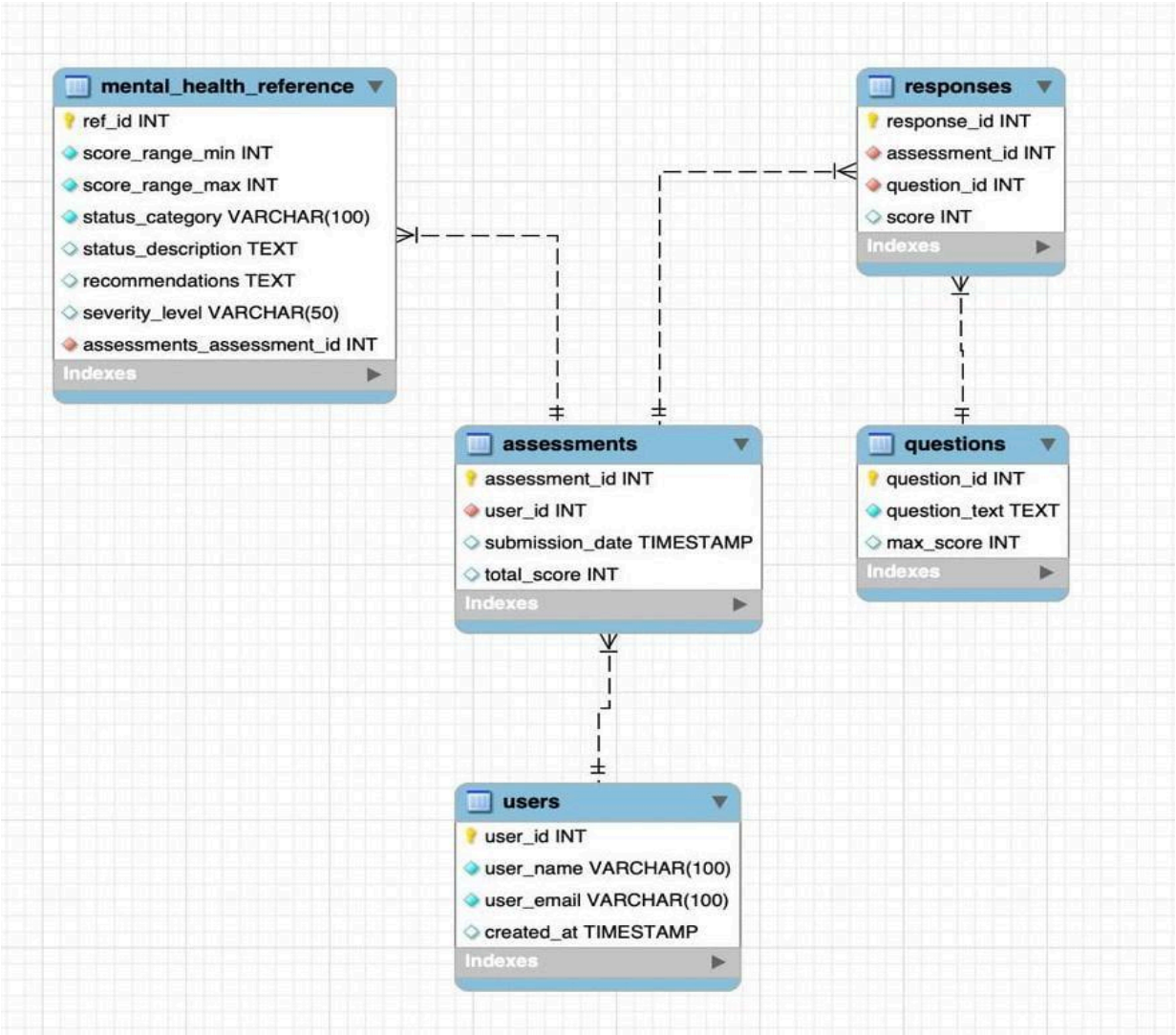

Figure 2.1: Relational Schema for Mental Health Assessment and Analysis System

2.2 Description of Tables

The tables used in the Mental Health Assessment and Analysis database along with their attributes, data types, size, and constraints are described below.

Table 2.1: User

- **Purpose:** Stores information about the user.
- **Primary Key:** id(Integer)
- **Data Types and Constraints:**

Field	Type	Null	Key	Default	Extra
user_id	int	NO	PRI	NULL	auto_increment
user_name	varchar(100)	NO		NULL	
user_email	varchar(100)	NO	UNI	NULL	
created_at	timestamp	YES		CURRENT	DEFAULT

Table 2.2: Responses

- **Purpose:** To collect the responses of the questions.
- **Primary Key:** id(Integer)
- **Data Types and Constraints:**

Field	type	Null	Key	Default	Extra
response_id	int	NO	PRI	NULL	auto_increment
assessment_id	int	NO	MUL	NULL	
question_id	int	NO	MUL	NULL	
score	int	YES		NULL	

Table 2.3: Questions

- **Purpose:** It contains the questions to be answered by the user.
- **Primary Key:** id(Integer)
- **Data Types and Constraints:**

Field	Type	Null	Key	Default	Extra
question_id	INT	NO	PRI	NULL	auto_increment
question_text	TEXT	NO		NULL	
max_score	INT	YES		**20**	

Table 2.4: Assessments

- **Purpose:** To calculate the score of the user.
- **Primary Key:** id(Integer)
- **Data Types and Constraints:**

Field	Type	Null	Key	Default	Extra
assessment_id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
submission_date	timestamp	YES		CURRENT	DEFAULT
total_score	int	YES		NULL	

Table 2.5: Mental Health Reference

- **Purpose:** It stores the reference values of the questions.
- **Primary Key:** id(Integer)
- **Data Types and Constraints:**

Field	Type	Null	Key	Default	Extra
ref_id	int	NO	PRI	NULL	auto_increment
score_min	int	NO		NULL	
score_max	int	NO		NULL	
status_category	varchar(100)	NO		NULL	
description	text	YES		NULL	
recommendations	text	YES		NULL	
severity_level	varchar(50)	YES		NULL	

2.3 Creation of Database and Tables - DDL Commands

- **CREATE DATABASE:**

```
CREATE DATABASE mental_health_tracker;
```

- **CREATE USER TABLE:**

```
CREATE TABLE users (
 user_id INT PRIMARY KEY AUTO_INCREMENT,
 user_name VARCHAR(100) NOT NULL,
user_email VARCHAR(100) UNIQUE NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- **CREATE QUESTIONS TABLE:**

```
CREATE TABLE questions (
question_id INT PRIMARY KEY AUTO_INCREMENT,
question_text TEXT NOT NULL,
```

```
max_score INT DEFAULT 20
);
```

- **CREATE ASSESSMENT TABLE:**

```
CREATE TABLE assessments (
assessment_id INT PRIMARY KEY AUTO_INCREMENT,
user_id INT NOT NULL,
submission_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 total_score INT,
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE CASCADE
);
```

- **CREATE RESPONSE TABLE:**

```
CREATE TABLE responses (
response_id INT PRIMARY KEY AUTO_INCREMENT,
assessment_id INT NOT NULL,
question_id INT NOT NULL,
score INT CHECK (score BETWEEN 1 AND 20),
FOREIGN KEY (assessment_id)
REFERENCES assessments(assessment_id)
ON DELETE CASCADE,
FOREIGN KEY (question_id)
REFERENCES questions(question_id)
ON DELETE CASCADE
```

```
);
```

- **CREATE REFERENCE TABLE:**

```
CREATE TABLE mental_health_reference (
ref_id INT PRIMARY KEY AUTO_INCREMENT,
score_range_min INT NOT NULL,
score_range_max INT NOT NULL,
status_category VARCHAR(100) NOT NULL,
status_description TEXT,
recommendations TEXT,
severity_level VARCHAR(50),
CHECK (score_range_min <= score_range_max)
);
```

2.4 Insertion of tuples into the table – DML commands

- **INSERT INTO REFERENCE:**

```
INSERT INTO mental_health_reference
(score_range_min, score_range_max, status_category, status_description,
recommendations, severity_level)
VALUES
(10, 60, 'Stable',
'You show minimal signs of stress or emotional distress.',
'Maintain healthy routines, exercise regularly, and continue positive habits.',
'Low'),
(61, 110, 'Mild Stress',
```

'You may be experiencing mild stress or emotional imbalance.',

'Practice mindfulness, improve sleep cycle, and monitor emotional changes.',

'Moderate');

- **INSERT INTO QUESTIONS:**

```
INSERT INTO questions (question_text) VALUES
('How often do you feel overwhelmed by daily responsibilities?'),
('How frequently do you experience difficulty sleeping due to stress?');
```

- **INSERT INTO USER:**

```
INSERT INTO users (user_name, user_email) VALUES
('Rahul Sharma', 'rahul@example.com'),
('Ananya Iyer', 'ananya@example.com');
```

- **INSERT INTO ASSESSMENT:**

```
INSERT INTO assessments (user_id, total_score) VALUES
(1, 135),
(2, 72);
```

- **INSERT INTO RESPONSE:**

```
INSERT INTO responses (assessment_id, question_id, score) VALUES
(1,1,15),(1,2,14),(1,3,13),(1,4,16),(1,5,12),
(1,6,14),(1,7,13),(1,8,15),(1,9,11),(1,10,12);
```

CHAPTER 3

Complex queries based on the concepts of constraints, sets, joins, views, Triggers and Cursors.

3.1 Adding Constraints and queries based on constraints

Question 1: Ensure that invalid scores (outside 1–20) are not inserted into the responses table.

SQL Statement: INSERT INTO responses (assessment_id, question_id, score) VALUES (1, 1, 25);

Output:

#	Time	Action
1	15:29:21	INSERT INTO responses (assessment_id, question_id, score) VALUES (1, 1, 25)

Figure 3.1.1: Error message indicating violation of CHECK constraint.

Question 2: Insert a valid response into the responses table.

SQL Statement: INSERT INTO responses (assessment_id, question_id, score) VALUES (1, 2, 15);

Output:

8	15:35:24	INSERT INTO responses (assessment_id, question_id, score) VALUES (1, 2, 15)

Figure 3.1.2: Record inserted successfully.

Question 3: Ensure that duplicate emails are not allowed in users table.

SQL Statement: INSERT INTO users (user_name, user_email) VALUES ('Test User', 'rahul@example.com');

Output:

Error Code: 1062. Duplicate entry 'rahul@example.com' for key 'users.user_email'

Figure 3.1.3: Error due to UNIQUE constraint violation.

3.2 Queries based on Aggregate Functions

Question 1: Find the average score of all assessments.

SQL Statement: SELECT AVG(total_score) AS average_score FROM assessments;

Output:

Result Grid | Filter

	average_score
▶	114.0000

Figure 3.2.1: Displays average total score

Question 2: Find the highest score achieved.

SQL Statement: SELECT MAX(total_score) AS max_score FROM assessments;

Output:

max_score
114

Figure 3.2.2: Displays highest score.

Question 3: Find the lowest score achieved.

SQL Statement: SELECT MIN(total_score) AS min_score FROM assessments;

Output:

Result Grid | Filter Rows:

min_score
114

Figure 3.2.3: Displays lowest total score.

3.3 Complex queries based on Sets

Question 1: Retrieve all user IDs from both users and assessments tables without duplicates.

SQL Statement: SELECT user_id FROM users UNION SELECT user_id FROM assessments;

Output:

Result Grid

	user_id
▶	2
	1

Figure 3.3.1: List of unique user IDs

Question 2: Retrieve all user IDs including duplicates.

SQL Statement: SELECT user_id FROM users UNION ALL SELECT user_id FROM assessments;

Output:

Result Grid

	user_id
▶	2
	1
	1
	2

Figure 3.3.2: List of user IDs including duplicates.

Question 3: Find common user IDs using logical comparison.

SQL Statement: SELECT user_id FROM users WHERE user_id IN (SELECT user_id FROM assessments);

Output:

Result Grid

user_id
2
1
NULL

Figure 3.3.3: List of user IDs present in both tables.

3.4 Complex queries based on Subqueries

Question 1: Find assessments with scores above average.

SQL Statement: SELECT * FROM assessments WHERE total_score > (SELECT AVG(total_score) FROM assessments);

Output:

Result Grid Filter Rows: Edit:

assessment_id	user_id	submission_date	total_score
NULL	NULL	NULL	NULL

Figure 3.4.1: Displays assessments with above-average scores.

Question 2: Find users who have taken assessments.

SQL Statement: SELECT user_name FROM users WHERE user_id IN (SELECT user_id FROM assessments);

Output:

Result Grid

	user_name
▶	Rahul Sharma
	Ananya Iyer

Figure 3.4.2: List of users who attempted the test.

Question 3: Find the assessment with the highest score.

SQL Statement: SELECT * FROM assessments WHERE total_score = (SELECT MAX(total_score) FROM assessments);

Output:

Result Grid | Filter Rows: | Edit:

	assessment_id	user_id	submission_date	total_score
▶	1	1	2026-03-21 08:29:24	124
*	NULL	NULL	NULL	NULL

Figure 3.4.3: Displays the highest scoring assessment.

3.5 Complex queries based on Joins

Question 1: Display user names with their total scores.

SQL Statement: SELECT u.user_name, a.total_score FROM users u JOIN assessments a ON u.user_id = a.user_id;

Output:

Result Grid | Filter Rows:

user_name	total_score
Rahul Sharma	124
Ananya Iyer	NULL

Figure 3.5.1: Shows user names and scores.

Question 2: Display user responses with question text.

SQL Statement: SELECT u.user_name, q.question_text, r.score FROM users u

JOIN assessments a ON u.user_id = a.user_id

JOIN responses r ON a.assessment_id = r.assessment_id

JOIN questions q ON r.question_id = q.question_id;

Output:

Result Grid | Filter Rows: | Export:

user_name	question_text	score
Rahul Sharma	Do you feel stressed frequently?	10
Rahul Sharma	Do you have trouble sleeping?	12
Rahul Sharma	Do you feel anxious without reason?	11
Rahul Sharma	Do you feel emotionally drained?	13
Rahul Sharma	Do you lose interest in activities?	9
Rahul Sharma	Do you feel irritated often?	10
Rahul Sharma	Do you struggle to concentrate?	11
Rahul Sharma	Do you feel hopeless?	12
Rahul Sharma	Do you experience mood swings?	10
Rahul Sharma	Do you avoid social interaction?	11
Rahul Sharma	Do you have trouble sleeping?	15
Ananya Iyer	Do you feel stressed frequently?	5
Ananya Iyer	Do you have trouble sleeping?	6
Ananya Iyer	Do you feel anxious without reason?	7
Ananya Iyer	Do you feel emotionally drained?	6
Ananya Iyer	Do you lose interest in activities?	5
Ananya Iyer	Do you feel irritated often?	6

Figure 3.5.2: Displays user responses for each question.

Question 3: Display assessment results with mental health category.

SQL Statement: SELECT a.assessment_id, a.total_score, m.status_category FROM assessments a
JOIN mental_health_reference m ON a.total_score BETWEEN m.score_range_min AND m.score_range_max;

Output:

	assessment_id	total_score	status_category
▶	1	124	Moderate Distress

Figure 3.5.3: Displays classification of each assessment.

3.6 Complex queries based on views

Question 1: Create a view for user scores.

SQL Statement: CREATE VIEW user_scores AS
SELECT u.user_name, a.total_score FROM users u
JOIN assessments a ON u.user_id = a.user_id;

Output: View created successfully.

Question 2: Retrieve data from the view.

SQL Statement: SELECT * FROM user_scores;

Output:

user_name	total_score
Rahul Sharma	124
Ananya Iyer	NULL

Figure 3.6.1: Displays user names and scores.

Question 3: Create a view for assessment classification.

SQL Statement: CREATE VIEW assessment_results AS
SELECT a.assessment_id, m.status_category FROM assessments a
JOIN mental_health_reference m ON a.total_score BETWEEN
m.score_range_min AND m.score_range_max;

Output: View created successfully.

3.7 Complex queries based on Triggers

Question 1: Automatically update total score when a response is inserted.

SQL Statement:

```
DELIMITER $$
CREATE TRIGGER calculate_total_score
AFTER INSERT ON responses
FOR EACH ROW
BEGIN
UPDATE assessments
SET total_score = ( SELECT SUM(score) FROM responses WHERE
assessment_id = NEW.assessment_id)
```

```
WHERE assessment_id = NEW.assessment_id;
END$$
DELIMITER ;
```

Output: Trigger created successfully.

Question 2: Insert response to test trigger.

SQL Statement: INSERT INTO responses (assessment_id, question_id, score) VALUES (1, 3, 12);

Output: Response inserted and total_score updated automatically.

Question 3: Check updated total score.
SQL Statement: SELECT total_score FROM assessments WHERE assessment_id = 1;

Output:

Result Grid

	total_score
▶	136

Figure 3.7.1: Updated total score displayed.

3.8 Complex queries based on Cursors

Question 1: Retrieve all user names one by one using a cursor.

SQL Statement:

```
DELIMITER $$
CREATE PROCEDURE get_all_users()
BEGIN
  DECLARE done INT DEFAULT FALSE;
  DECLARE uname VARCHAR(100);
  DECLARE cur CURSOR FOR
     SELECT user_name FROM users;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done =
TRUE;
  OPEN cur;
  read_loop: LOOP
     FETCH cur INTO uname;
     IF done THEN
        LEAVE read_loop;
     END IF;
     SELECT uname AS user_name;
  END LOOP;
  CLOSE cur;
END$$
DELIMITER ;
CALL get_all_users();
```

Output:

Result Grid | Filter R

	user_name
▶	Ananya Iyer

Figure 3.8.1: List of user names displayed one by one.

Question 2: Retrieve all assessment scores using a cursor.

SQL Statement: CALL get_all_scores();

Output: All assessment scores displayed sequentially.

Question 3: Retrieve user names along with their scores using a cursor.

SQL Statement: CALL get_user_scores_cursor();

Output: Displays user names along with their total scores one by one.